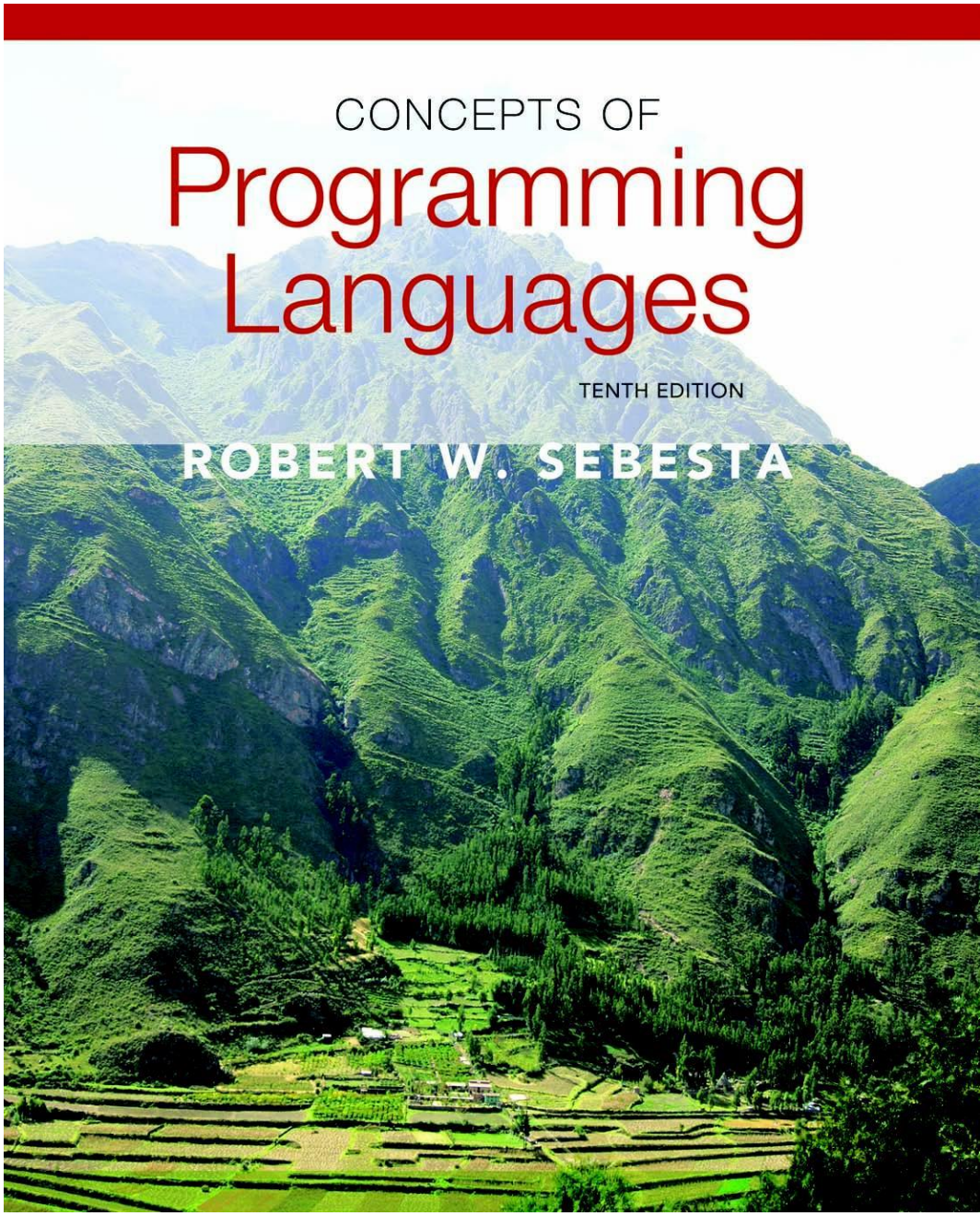
Chapter 16

Logic Programming Languages



Chapter 16 Topics

- Introduction
- A Brief Introduction to Predicate Calculus
- Predicate Calculus and Proving Theorems
- An Overview of Logic Programming
- The Origins of Prolog
- The Basic Elements of Prolog
- Deficiencies of Prolog
- Applications of Logic Programming

Copyright © 2012 Addison-Wesley. All rights reserved.

Introduction

- Programs in logic languages
  - are expressed in a form of symbolic logic
  - use a logical inferencing process to produce results
  - *Declarative* rather than *procedural*:
    - Only specification of *results* are stated (not detailed *procedures* for producing them)
  - Are collections of facts and rules

Copyright © 2012 Addison-Wesley. All rights reserved.

Predicate Calculus

- A **proposition** can be thought as a logical statement that may or may not be true
  - Consists of objects and relationships of objects to each other
- **Symbolic logic** which can be used for the basic needs of formal logic:
  - Express propositions
  - Express relationships between propositions
  - Describe how new propositions can be inferred from other propositions
- The particular form of symbolic logic used for logic programming called ***predicate calculus***

Copyright © 2012 Addison-Wesley. All rights reserved.

Object Representation in Propositions

- Objects in propositions are represented by simple terms: either constants or variables
- *Constant*: a symbol that represents an object
- *Variable*: a symbol that can represent different objects at different times
  - Different from variables in imperative languages

Copyright © 2012 Addison-Wesley. All rights reserved.

Compound Terms

- *Atomic propositions* consist of compound terms

- *Compound term*: one element of a mathematical relation, written like a mathematical function
  - Mathematical function is a mapping
  - Can be written as an expression, as a table or list of tuples

Copyright © 2012 Addison-Wesley. All rights reserved.

Parts of a Compound Term

- Compound term composed of two parts
  - Functor: function symbol that names the relationship
  - Ordered list of parameters (tuple)

- Examples:

`student(john)`	1-tuple
`like(seth, OSX)`	2-tuple
`like(nick, Windows)`	2-tuple
`like(jim, Linux)`	2-tuple

Copyright © 2012 Addison-Wesley. All rights reserved.

Forms of a Proposition

- Propositions can be stated in two forms:
  - *Fact*: proposition is assumed to be true
  - *Query*: truth of proposition is to be determined

- Compound propositions:
  - Have two or more atomic propositions
  - Propositions are connected by logical connectors, or operators

Copyright © 2012 Addison-Wesley. All rights reserved.

Logical Operators

Name	Symbol	Example	Meaning
negation	$\neg$	$\neg$ a	not a
conjunction	$\cap$	a $\cap$ b	a and b
disjunction	$\cup$	a $\cup$ b	a or b
equivalence	$\equiv$	a $\equiv$ b	a is equivalent to b
implication	$\supset$ $\subset$	a $\supset$ b a $\subset$ b	a implies b b implies a

Copyright © 2012 Addison-Wesley. All rights reserved.

Example Compound Propositions

- The $\neg$ operator has the highest precedence.
- The operators $\cap$, $\cup$, and $\equiv$ all have higher precedence than $\supset$ and $\subset$.

1. $\mathbf{a \cap b \supset c}$ is equivalent to $\mathbf{(a \cap b) \supset c}$
2. $\mathbf{a \cap \neg b \supset d}$ is equivalent to $\mathbf{(a \cap (\neg b)) \supset d}$

Copyright © 2012 Addison-Wesley. All rights reserved.

Quantifiers

- Variables can appear in propositions but only when introduced by special symbols called ***quantifiers***.
- Predicate calculus includes two quantifiers
- X is a variable and P is a proposition

Name	Example	Meaning
universal	$\forall X.P$	For all X, P is true
existential	$\exists X.P$	There exists a value of X such that P is true

Copyright © 2012 Addison-Wesley. All rights reserved.

Quantifiers

- The period between X and P simply separates the variable from the proposition.

1. ∀X. (**woman (X) ⊃ human (X)**) → means that for any value of X, if X is a woman, then X is a human.

2. ∃X. (**mother (mary, X) ∩ male (X)**) → means that there exists a value of X such that mary is the mother of X and X is a male; in other words, mary has a son.

the quantifiers have higher precedence than any of the operators

Copyright © 2012 Addison-Wesley. All rights reserved.

Clausal Form

- In predicate calculus, there are too many ways to state the same thing
- We need to use a standard form for propositions: ***Clausal form***
  - $B_1 \cup B_2 \cup \ldots \cup B_n \subset A_1 \cap A_2 \cap \ldots \cap A_m$
  - means if all the As are true, then at least one B is true
    - ($A_1 \cap A_2 \cap \ldots \cap A_m$) implies ($B_1 \cup B_2 \cup \ldots \cup B_n$)

Copyright © 2012 Addison-Wesley. All rights reserved.

Clausal Form Characteristics

- Existential quantifiers are not required
- Universal quantifiers are implicit in the use of variables in the atomic propositions
- no operators other than conjunction and disjunction are required
  - appear only in the order shown in the general clausal form:
    - disjunction on the left side and conjunction on the right side.
    - Antecedent: right side of a clausal form
    - Consequent: left side of a clausal form

Copyright © 2012 Addison-Wesley. All rights reserved.

Clausal Form Propositions

1. `likes (bob, trout) ⊂ likes (bob, fish) ∩ fish (trout)`

   - if bob likes fish and a trout is a fish, then bob likes trout.

2. `father (louis, al) ∪ father (louis, violet) ⊂`
   `father (al, bob) ∩ mother (violet, bob) ∩`
   `grandfather (louis, bob)`

   - if al is bob's father and violet is bob's mother and louis is bob's grandfather, then louis is either al's father or violet's father.

Copyright © 2012 Addison-Wesley. All rights reserved.

Predicate Calculus and Proving Theorems

- A use of collections of propositions is to discover new theorems that can be inferred from them

- *Resolution*: an inference principle that allows inferred propositions to be computed from given propositions
  - provides a method with potential application to automatic theorem proving

Copyright © 2012 Addison-Wesley. All rights reserved.

Concept of Resolution

- Suppose there are two propositions with the forms:

  P1 ⊂ **P2** → P2 implies P1

  Q1 ⊂ **Q2** → Q2 implies Q1

- Suppose that P1 is identical to Q2, so that we could rename P1 and Q2 as T:

  T ⊂ **P2** → P2 implies T

  Q1 ⊂ **T** → T implies Q1

  →it is logically obvious that P2 implies Q1, which we could write as:

  Q1 ⊂ **P2**

The process of inferring this proposition from the original two propositions is **resolution.**

Copyright © 2012 Addison-Wesley. All rights reserved.

Example Resolution

```
older (joanne, jake) ⊂ mother (joanne, jake)
wiser (joanne, jake) ⊂ older (joanne, jake)
```

- From these propositions, the following proposition can be constructed using resolution:

```
wiser(joanne, jake) ⊂ mother(joanne, jake)
```

- left sides are OR'ed together to make the left side of the new proposition.
- right sides are AND'ed together to get the right side of the new proposition.
- any term that appears on both sides of the new proposition is removed from both sides.

Copyright © 2012 Addison-Wesley. All rights reserved.

Example Resolution

$$\text{father(bob,jake)} \cup \text{mother(bob,jake)} \subset \text{parent(bob,jake)}$$

$$\text{grandfather(bob,fred)} \subset \text{father(bob,jake)} \cap \text{father(jake,fred)}$$

resolution says that

$$\text{father(bob,jake)} \cup \text{mother(bob,jake)} \cup \text{grandfather(bob,fred)} \subset \text{parent(bob,jake)} \cap \text{father(bob,jake)} \cap \text{father(jake,fred)}$$

finally

$$\text{mother(bob,jake)} \cup \text{grandfather(bob,fred)} \subset \text{parent(bob,jake)} \cap \text{father(jake,fred)}$$

Copyright © 2012 Addison-Wesley. All rights reserved.

Example Resolution

$$\texttt{father(bob,jake)} \cup \texttt{mother(bob,jake)} \subset \texttt{parent(bob,jake)}$$

$$\texttt{grandfather(bob,fred)} \subset \texttt{father(bob,jake)} \cap \texttt{father(jake,fred)}$$

$$\texttt{mother(bob,jake)} \cup \texttt{grandfather(bob,fred)} \subset \texttt{parent(bob,jake)} \cap \texttt{father(jake,fred)}$$

if: bob is the parent of jake implies that bob is either the father or mother of jake

and: bob is the father of jake and jake is the father of fred implies that bob is the grandfather of fred

then: *if* bob is the parent of jake and jake is the father of fred *then:* either bob is jake's mother or bob is fred's grandfather

Copyright © 2012 Addison-Wesley. All rights reserved.

Resolution

- *Unification*: finding values for variables in propositions that allows matching process to succeed
- *Instantiation*: assigning temporary values to variables to allow unification to succeed

- After instantiating a variable with a value, if matching fails, may need to *backtrack* and instantiate with a different value

Copyright © 2012 Addison-Wesley. All rights reserved.

Proof by Contradiction

- An important property of resolution is its ability to detect any inconsistency in a given set of propositions.

- *Hypotheses*: a set of propositions
- *Goal*: negation of theorem stated as a proposition

- Theorem is proved by finding an inconsistency

Copyright © 2012 Addison-Wesley. All rights reserved.

Theorem Proving

- Basis for logic programming
- Resolution on a hypotheses and a goal may require large amount of time
- When propositions used for resolution, only restricted form can be used
- *Horn clause* – can have only two forms
  - *Headed*: single atomic proposition on left side
  - *Headless*: empty left side (used to state facts)
- Most propositions can be stated as Horn clauses

Copyright © 2012 Addison-Wesley. All rights reserved.

Horn Clauses

- Headed Horn clause:

```
likes(bob, trout) ⊂ likes(bob, fish) ∩ fish(trout)
```

- Headless Horn clause:

```
father(bob, jake)
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Overview of Logic Programming

- Declarative semantics
  - There is a simple way to determine the meaning of each statement
  - Simpler than the semantics of imperative languages
    - the meaning of a proposition in a logic PL can be determined from the statement itself.
    - the semantics of a simple assignment statement in imperative PL requires: examination of local declarations, knowledge of the scoping rules, and examination of programs in other files just to determine the types of the variables
- Programming is nonprocedural
  - Programs do not state how a result is to be computed, but rather the form of the result

Copyright © 2012 Addison-Wesley. All rights reserved.

Example: Sorting a List

- Describe the characteristics of a sorted list, not the process of rearranging a list

- `sort(old_list, new_list)`
  `⊂ permute (old_list, new_list) ∩ sorted (new_list)`
- `sorted (list)`
  `⊂ ∀j such that 1 ≤ j <n, list(j) ≤ list(j+1)`

– permute is a predicate that returns true if its second parameter array is a permutation of its first parameter array.

Copyright © 2012 Addison-Wesley. All rights reserved.

The Origins of Prolog

- University of Aix-Marseille (Calmerauer & Roussel)
  - Natural language processing
- University of Edinburgh (Kowalski)
  - Automated theorem proving

Copyright © 2012 Addison-Wesley. All rights reserved.

Terms

- This book uses the Edinburgh syntax of Prolog
- *Term*: a constant, variable, or structure

1. **Constant**: an atom or an integer
   - *Atom*: symbolic value of Prolog
   - Atom consists of either:
     - a string of letters, digits, and underscores beginning with a lowercase letter
     - a string of printable ASCII characters delimited by apostrophes

Copyright © 2012 Addison-Wesley. All rights reserved.

Terms: Variables and Structures

2. **Variable**: any string of letters, digits, and underscores beginning with an uppercase letter or underscore(_)

- *Instantiation*: binding of a variable to a value
  - Lasts only as long as it takes to satisfy one complete goal

Copyright © 2012 Addison-Wesley. All rights reserved.

Terms: Variables and Structures

3. **Structure**: represents atomic propositions of predicate calculus. General form:

   `functor(`*`parameter list`*`)`

   - The functor is any atom and is used to identify the structure.
   - The parameter list can be any list of atoms, variables, or other structures.

- Structures are means of specifying facts in Prolog.
- They can be thought of as objects, in which case they allow facts to be stated in terms of several related atoms.
- In this sense, structures are relations – they state relationships among terms.
- A structure is also a predicate when its context specifies it to be a query (question).

Copyright © 2012 Addison-Wesley. All rights reserved.

Fact Statements

- Prolog statements are used to construct the hypotheses, or database of assumed information
- Prolog has two basic statement forms
  - headless Horn clauses
    - simplest form in Prolog is a single structure, which is interpreted as an unconditional assertion or fact. facts are simply propositions that are assumed to be true
    - Examples:

```
female(shelley).
male(bill).
father(bill, jake).
```

  - headed Horn clauses

Copyright © 2012 Addison-Wesley. All rights reserved.

Rule Statements

- Other basic form of Prolog statement for constructing the hypotheses or database
- Headed Horn clause:

```
consequence :- antecedent_expression.
ancestor(mary,shelley) :- mother(mary,shelley).
```

  - Right side: *antecedent* (*if* part)
    - May be single term or conjunction
  - Left side: *consequent* (*then* part)
    - Must be single term

- *Conjunction*: multiple terms separated by logical AND operations (implied)

```
female(shelley), child(shelley).
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Example Rules

- Prolog statements can use variables to generalize meaning:

```
parent(X,Y):- mother(X,Y).
parent(X,Y):- father(X,Y).
grandparent(X,Z):- parent(X,Y), parent(Y,Z).
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Goal Statements

- For theorem proving, theorem is in form of proposition that we want system to prove or disprove – *goal statement*
- Same format as headless Horn

```
man(fred).
```

- Conjunctive propositions and propositions with variables also legal goals

```
father(X, mike)
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Inferencing Process of Prolog

- Queries are called goals
- If a goal is a compound proposition, each of the facts is a subgoal
- To prove a goal is true, must find a chain of inference rules and/or facts. For goal Q:

  P_2 :- P_1

  P_3 :- P_2

  ...

  Q :- P_n
- Process of proving a subgoal called matching, satisfying, or resolution

Copyright © 2012 Addison-Wesley. All rights reserved.

Approaches

- *Matching* is the process of proving a proposition
- Proving a subgoal is called *satisfying* the subgoal
- *Bottom-up resolution, forward chaining*
  - Begin with facts and rules of database and attempt to find sequence that leads to goal
  - Works well with a large set of possibly correct answers
- *Top-down resolution, backward chaining*
  - Begin with goal and attempt to find sequence that leads to set of facts in database
  - Works well with a small set of possibly correct answers
- Prolog implementations use backward chaining

Copyright © 2012 Addison-Wesley. All rights reserved.

Subgoal Strategies

- When goal has more than one subgoal, can use either
  - Depth-first search: find a complete proof for the first subgoal before working on others
  - Breadth-first search: work on all subgoals in parallel
- Prolog uses depth-first search
  - Can be done with fewer computer resources

Copyright © 2012 Addison-Wesley. All rights reserved.

Backtracking

- With a goal with multiple subgoals, if fail to show truth of one of subgoals, reconsider previous subgoal to find an alternative solution: *backtracking*
- Begin search where previous search left off
- Can take lots of time and space because may find all possible proofs to every subgoal

Copyright © 2012 Addison-Wesley. All rights reserved.

Simple Arithmetic

- Prolog supports integer variables and integer arithmetic
- `is` operator: takes an arithmetic expression as right operand and variable as left operand

```
A is B / 17 + C
```

- Not the same as an assignment statement!
  - The following is illegal:

```
Sum is Sum + Number.
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Example

```
speed(ford,100).
speed(chevy,105).
speed(dodge,95).
speed(volvo,80).
time(ford,20).
time(chevy,21).
time(dodge,24).
time(volvo,24).
distance(X,Y) :-   speed(X,Speed),
                      time(X,Time),
                      Y is Speed * Time.
```

A query: `distance(chevy, Chevy_Distance).`

Copyright © 2012 Addison-Wesley. All rights reserved.

Trace

- Built-in structure that displays instantiations at each step
- *Tracing model* of execution – four events:
  - **Call** (occurs at the beginning of attempt to satisfy goal)
  - **Exit** (occurs when a goal has been satisfied)
  - **Redo** (occurs when backtrack causes an attempt to resatisfy a goal)
  - **Fail** (occurs when goal fails)

Copyright © 2012 Addison-Wesley. All rights reserved.

Example

```
speed(ford,100).
speed(chevy,105).
speed(dodge,95).
speed(volvo,80).
time(ford,20).
time(chevy,21).
time(dodge,24).
time(volvo,24).
distance(X,Y) :- speed(X,Speed),
                 time(X,Time),
                 Y is Speed * Time.
```

```
trace.
distance(chevy, Chevy_Distance).
(1) 1 Call: distance(chevy, _0)?
(2) 2 Call: speed(chevy, _5)?
(2) 2 Exit: speed(chevy, 105)
(3) 2 Call: time(chevy, _6)?
(3) 2 Exit: time(chevy, 21)
(4) 2 Call: _0 is 105*21?
(4) 2 Exit: 2205 is 105*21
(1) 1 Exit: distance(chevy, 2205)
Chevy_Distance = 2205
```

- Symbols in the trace that begin with the underscore character (_) are internal variables used to store instantiated values.
- The first column of the trace indicates the subgoal whose match is currently being attempted (3) – instantiate the temporary variable _6 with a time value for chevy)
- The second column indicates the call depth of the matching process.
- The third column indicates the current action.

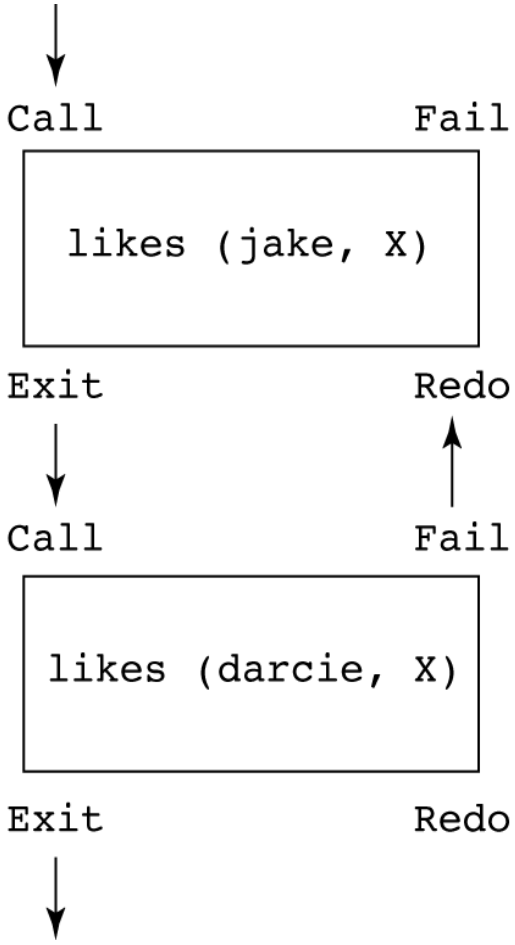
Copyright © 2012 Addison-Wesley. All rights reserved.

Example

```
likes(jake,chocolate).
likes(jake, apricots).
likes(darcie, licorice).
likes(darcie, apricots).
```

```
trace.
likes(jake, X), likes(darcie, X).
(1) 1 Call: likes(jake, _0)?
(1) 1 Exit: likes(jake, chocolate)
(2) 1 Call: likes(darcie, chocolate)?
(2) 1 Fail: likes(darcie, chocolate)
(1) 1 Redo: likes(jake, _0)?
(1) 1 Exit: likes(jake, apricots)
(3) 1 Call: likes(darcie, apricots)?
(3) 1 Exit: likes(darcie, apricots)
X = apricots
```

Consider each goal as a box with four ports—call, fail, exit, and redo



Copyright © 2012 Addison-Wesley. All rights reserved.

List Structures

- Other basic data structure (besides atomic propositions we have already seen): **`list`**
- *List* is a sequence of any number of elements
  - Elements can be atoms, atomic propositions, or other terms (including other lists)

```
[apple, plum, grape, kumquat]
[]          (empty list)
[X | Y]     (head X and tail Y)
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Append Example

```
append([], List, List).
append([Head | List_1], List_2, [Head | List_3]) :-
              append (List_1, List_2, List_3).
```

```
trace.
append([bob, jo], [jake, darcie], Family).
(1) 1 Call: append([bob, jo], [jake, darcie], _10)?
(2) 2 Call: append([jo], [jake, darcie], _18)?
(3) 3 Call: append([], [jake, darcie], _25)?
(3) 3 Exit: append([], [jake, darcie], [jake, darcie])
(2) 2 Exit: append([jo], [jake, darcie], [jo, jake, darcie])
(1) 1 Exit: append([bob, jo], [jake, darcie], [bob, jo, jake, darcie])
Family = [bob, jo, jake, darcie]
```

Copyright © 2012 Addison-Wesley. All rights reserved.

More Examples

```
reverse([], []).
reverse([Head | Tail], List) :-
       reverse (Tail, Result), append (Result, [Head], List).
```

```
member(Element, [Element | _]).
member(Element, [_ | List]) :- member(Element, List).
```

The underscore character means an anonymous variable—it means we do not care what instantiation it might get from unification

Copyright © 2012 Addison-Wesley. All rights reserved.

Applications of Logic Programming

- Relational database management systems
- Expert systems
- Natural language processing

Copyright © 2012 Addison-Wesley. All rights reserved.

Summary

- Symbolic logic provides basis for logic programming
- Logic programs should be nonprocedural
- Prolog statements are facts, rules, or goals
- Resolution is the primary activity of a Prolog interpreter
- Although there are a number of drawbacks with the current state of logic programming it has been used in a number of areas

Copyright © 2012 Addison-Wesley. All rights reserved.